

Explainable AI for Understanding Network and System Behavior

Andreas Marelius Stachnik Hansen
IN3260 Bachelor Project

May 15, 2026

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Problem Definition	3
1.3	Objectives	3
1.4	Report Structure	3
2	Background	4
2.1	Anomaly Detection in Monitoring Systems	4
2.2	Statistical Detection via Rolling Z-Score	4
2.3	Scikit learn	4
2.3.1	Fitting a model	4
2.4	Bias	4
3	Synthetic Data Generation	5
3.1	Design Principles	5
3.2	Monitoring Metrics	5
3.2.1	Metrics to add	6
3.3	Daily Behavior Modeling	6
3.4	Latent Event Modeling	6
3.5	Event Types	7
3.6	Baseline Anomaly Detection	7
3.6.1	Detection Rule	7
3.6.2	Interval Aggregation	7
3.6.3	Interval-Level Explanation	7
4	Machine Learning Models	7
4.1	Logistic Regression	7
4.1.1	Pipeline	8
4.2	Random Forest	9
4.3	Neural Network	9
4.3.1	Activation functions	10
4.3.2	The model for this project	10
4.4	Training Setup	10

4.5	Performance Results	11
4.5.1	Problems encountered	11
4.5.2	Too high scores	12
4.5.3	F1 score	12
4.5.4	Precision	12
4.5.5	Recall	12
4.5.6	AUC	13
5	Explainable AI Methods	14
5.1	The need for explainable AI	14
5.2	Permutation Feature Importance	14
5.3	SHAP	15
5.4	LIME	17
5.5	Integrated Gradients	18
5.6	Strengths and weaknesses	19
6	Evaluation Methodology	19
6.1	Top-k Overlap	19
6.2	Driver Recovery@k	19
6.3	Stability Under Noise	19
7	Experimental Results	19
7.0.1	Models used for testing	19
7.1	Global Importance Comparison	20
7.2	Local Explanation Analysis	21
7.3	Driver Recovery Results	21
7.3.1	Pre processing	21
7.3.2	Results	22
8	Real life data	22
8.1	From synthetic to real data	22
8.2	Comparison of data	23
8.3	Real data experiment	23
8.3.1	The initial test with models	23
8.3.2	The explainable AI setup	23
8.3.3	The results	24
8.3.4	The results for random forest	24
8.3.5	The results for logistic regression	24
8.3.6	Conclusion for test	26
9	Conclusion	26

1 Introduction

1.1 Motivation

Modern network infrastructures continuously generate multivariate monitoring data that describe the health of the system and network. Although anomaly detection systems can identify abnormal patterns, they often operate as black boxes, providing limited insight into the causes of alerts. In operational settings, understanding the drivers of anomalies is essential for root-cause analysis, mitigation, and trust.

Explainable AI can give powerful insight for not only people who have great knowledge within the field of AI, but it could also be used to help give a greater understanding and transparency for people who do not have a great deal of knowledge within the field of AI.

In this age of rapid development of artificial development, there need to be a way for users to gain insight of how a model came to a conclusion. To understand how a model works is to gain trust that a model is not misleading or manipulating.

1.2 Problem Definition

This project investigates the following research question:

Can explanation methods reliably identify the true drivers of anomalous behavior in multivariate network monitoring data?

We address this problem in a controlled synthetic environment where ground-truth drivers are known. With a small experiment with real data at the end to test the reliability.

1.3 Objectives

The objectives are:

- Design a realistic synthetic network monitoring dataset
- Implement a statistical anomaly detection baseline
- Train supervised models for anomaly classification
- Apply multiple XAI methods
- Quantitatively compare explanation correctness

1.4 Report Structure

The remainder of this report is organized as follows: Background theory is presented in Section 2. Synthetic data generation is described in Section 3. Baseline detection is introduced in Section 4. Machine learning models are detailed in Section 5. Explanation methods are discussed in Section 6. Evaluation methodology is defined in Section 7. Experimental results are shown in Section 8. A real data experiment in Section 9. Discussion and conclusions follow.

2 Background

2.1 Anomaly Detection in Monitoring Systems

An anomaly represents a deviation from expected behavior. In network monitoring, anomalies may correspond to congestion, hardware faults, overload events, or misconfigurations.

2.2 Statistical Detection via Rolling Z-Score

Given a time-series x_t , the rolling z-score is defined as:

$$z_t = \frac{x_t - \mu_t}{\sigma_t} \quad (1)$$

where μ_t and σ_t are computed over a sliding window.

This method is interpretable and computationally efficient, making it suitable as a baseline.

The rolling z score is a efficient way to detect events and discover scores out of the ordinary.

The way it works is that the areas with higher scores get flagged as these have deviated from the standard.

2.3 Scikit learn

One of the biggest libraries for training and evaluating artificial intelligence for python. To train an AI from scratch can be error prone and can take time, but Scikit have multiple useful models and algorithms ready to use. Some examples of these are the random forest, logistic regression and neural networks. Most of the explainable AI methods work well with the models made from this library. This makes it ideal for testing the different explainable AI methods.

2.3.1 Fitting a model

From the observed training data we create a line to correctly classify the features put into the model. The easiest and most basic way to fit the model is via the least square method.

An example of fitting could be when we have person who observes some sort of natural phenomenon. The person will gain an insight and form rules for why the phenomenon happens based on the his or hers observaitons. They have fitted rules from a set of features, which is what a model also does.

2.4 Bias

Bias can arise from a number of reasons. These biases can effect the performance of an AI model negatively as it hinders the models in being objective. The most common bias for this type of project that creates its own dataset is the "experimenter's bias". This happens when a researcher wants to confirm their hypotheses so they create data to train their AI model specifically to solve this hypothesis, and do not take into consideration

anything else. This is why the dataset is trying to be so standard as possible with the equation provided in Section 3.3 with added noise and events randomly inserted.

3 Synthetic Data Generation

3.1 Design Principles

The dataset is designed to replicate realistic monitoring data:

- Daily usage cycles
- Random noise
- Correlated metrics
- Finite-duration events

The random noise can be set to be more intense to test how good the models perform.

3.2 Monitoring Metrics

The dataset includes: Latency, Throughput, Packet Loss, Jitter, CPU Usage, Memory Usage, and I/O Delay.

The latency is in the round trip time for a package. This is a basic metric that gives good insight to how the network performs, and what problems might arise. If the latency is low then the network performs well. A high latency could be an indicator of a congestion event or bad flow control, which could give us problems in the future where packets do not arrive at all.

The throughput is the maximum throughput. The need for a maximum throughput comes from the users perspective. As the user might never reach any critical speeds, it would then be hard to detect the any problems through the current throughput alone. The monitoring happens every minute as per the synthetic data set, but if the monitoring happend at shorter intervals, then it would need to be the current throughput. This is a simple difference in this synthetic training set and what would happen in a real monitoring situation.

Packet loss is measured in percentage as is the standard. The more packets that are lost, the bigger the performance hit. A missing packet will require resending our loss off data leading to trouble for the user either talking over live stream or playing video games.

Jitter is the differences in the arrival time. Jitter can cause packets to arrive at different times, which could be detrimental to a connections that need a stable access, like video calls and online video games. With bigger jitter values the packets might not arrive in order and could in some cases not be valuable to the connection anymore.

CPU usage is the percentage of the total CPU used. The CPU the most central part of the pc and higher percentages could be an indicator of too much work . This means that the CPU could be damaged over time.

Memory usage is the memory used in percentage. High memory could lead to a lot of swapping which then uses the disk more. This could be bad, but there are often cases where modern operating systems will use a lot of memory for caches. This is more of a sign of efficiency than being slow [4]. So a better metric might be to look at total number

of processes or threads plus total memory usage as this provide insight to wether it is high usage due to optimization or context switches.

I/O delay is a metric that could be changed. It does its job when it comes to the synthetic dataset, but it is harder when it comes to the real world. Most computers have different delay when it comes to I/O. There would be a notable difference between computers using an SSD, and one which uses disks. It might also be a bit ambiguous as I/O can mean so much in a computer from the network interface card to . For this project the I/O delay is the delay it takes to write/ read to disk.

3.2.1 Metrics to add

One metric to add would be GPU percentage usage. The gpu is a vital part of modern computers, and have in the rise of AI started to become a vital part of data centers. For the average gamer it could be of great help to check wether a fault or a bad event was due to the CPU, GPU or something entirely different.

3.3 Daily Behavior Modeling

Normal behavior is modeled as:

$$\mu(t) = 0.5 + 0.5 \sin\left(\frac{2\pi t}{T}\right) \quad (2)$$

This equation follows a normal day of activity. It starts with a slope upward, where it reaches its main peak activity in the middle of the day, before it slopes down for the rest of the day. This is what a typical user at company can experience or a data center, where the midday is peak usage. There is also a gradual increase and gradual decrease instead of rapid changes. This fits for a time span of one day, but might have been changed out if this was based on a shorter time span or if it was based on a personal computer.

These datasets would not work for the average computer user. In this dataset the metrics all follow the curve. A normal computer at home would not use all available resources at once all the time. If the user were to for example stream a video, then the networking metrics would fluctuate, but the metrics of I/O delay would not be used and the others would increase depending on how big the video packets are. One of the few reasons that the I/O delay would increase is during the downloading of large files, or playing video games with large asset files. This is also a point where all of the other metrics would increase if the game was a multiplayer game. It then could be said that the best use case of these classification models and the explainable AI for the average person is when playing games.

3.4 Latent Event Modeling

Events are injected using a latent factor C_t :

$$Latency_t = \mu_L(t) + \alpha_1 C_t + \epsilon_L \quad (3)$$

$$Throughput_t = \mu_T(t) - \alpha_2 C_t + \epsilon_T \quad (4)$$

$$CPU_t = \mu_C(t) + \alpha_3 C_t + \epsilon_C \quad (5)$$

This structure creates realistic cross-metric correlation.

3.5 Event Types

- Congestion
- CPU saturation
- I/O bottleneck

These are some of the most common events that a pc will encounter during runtime. It is of great importance to be able to detect these, but also be able to show how a black box model attribute to these events. To be able to detect these problems will give a useful insight to both normal consumers who play video games or stream videos, and to professionals who work with powerful computers in datacenters or other industries. There are of course other important events, but these are easy to create data for, and reason about.

3.6 Baseline Anomaly Detection

This was used as proof of concept. This proved that easy statistics could solve the easy problems that relate to detection and anomaly detection. The original models used this as training data originally, but was phased out over having some issues with some lacking detections. This caused the models to not be able to detect properly the actual events properly.

3.6.1 Detection Rule

A time step is flagged if:

$$|z_{m,t}| \geq \tau \quad (6)$$

for at least k metrics.

3.6.2 Interval Aggregation

Consecutive anomaly points are grouped into intervals.

3.6.3 Interval-Level Explanation

Metric contribution score:

$$Score_m = \frac{1}{T} \sum_{t \in interval} |z_{m,t}| \quad (7)$$

4 Machine Learning Models

4.1 Logistic Regression

Linear baseline classifier. This method classifies between two classes. The output value of this model can only be between 2 values, 0 and 1. This is a difference from the linear regression that maps to a metric value such as prices for gas. The logistic value is therefore a value between 0 and 1. The power of logistic regression is that the features can span

from $-\infty$ to $+\infty$ and the result will still be between 0 to 1. In the case of this project the bigger the logistic regression output value the bigger chance of an event happening. Where above a value of 0.5 or 50% the instance is flagged as an event. This could be adjusted if needed.

To summarize the goal of logistic regression: It is to assign a probability of a classification. We look at all the input variables and from these we decide a likely outcome.

This diagram shows a simple logistic regression model. The students have practiced from 0-6 hours and either passed or failed their exam. There is only one feature. The dots are the data used in fitting the model. And the curve is the model.

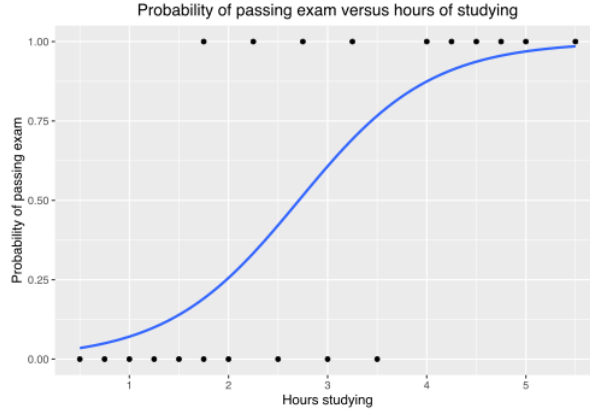


Figure 1: Simple logistic regression model

[15]

The basic understanding from statistics:

$$f(z) = \frac{1}{1 + e^{-z}} \quad (8)$$

Here $f(z)$ is the logistic function with z as the linear regression:

$$z = \hat{y} = b_1x_1 + b_2x_2 + \dots + b_kx_k + a \quad (9)$$

To get the probability of an event we then use this equation.

$$P(is\ an\ event) = \frac{1}{1 + e^{-(b_1*throughput+b_2*latency...+a)}} \quad (10)$$

The b_n are the weights assigned by machine learning or rather through the so called maximum likelihood method in statistics. a is a constant also learned through this method. [1]

The logistic regression will not be able to see a difference between events, but will be able to tell if it is an event or not. It is in other words a binary classification.

4.1.1 Pipeline

A systematic way of dividing the necessary steps to build an ML. For training this model A pipeline was used.

4.2 Random Forest

Ensemble of decision trees. The random forest uses multiple trees that in the end come to a decision on what result will be. For our use case, we use random forest classification instead of random forest regression. The main difference is in how a model is chosen, in classification we use the result that most of the trees agreed on, and when we use regression we use the average decision.

The main problem with decision trees are that they tend to overfit. This means that they can reason about very specific cases. This also means that there is low bias, but high variance. The random forest counteract the high variance by allowing votes on the best result ensuring the outliers of the vote do not get to sway the result. The negative side effect of random forrest is that we lose the interpretability of decision trees. [14]

The random forest is made up of decision trees. These trees will recursively build until only pure leaf nodes are constructed. A pure leaf node is one where a node only contains one class(Non-event/event).

These trees look like pure if statements with decision nodes and results in the leaf nodes. It is machine learning because of how it reaches these if statements. The trees will seek the greatest information gain for every level in the decision tree. When the information gain is great, the entropy is low(the number of classes). This is done by going through every feature to get the best entropy result. [9]

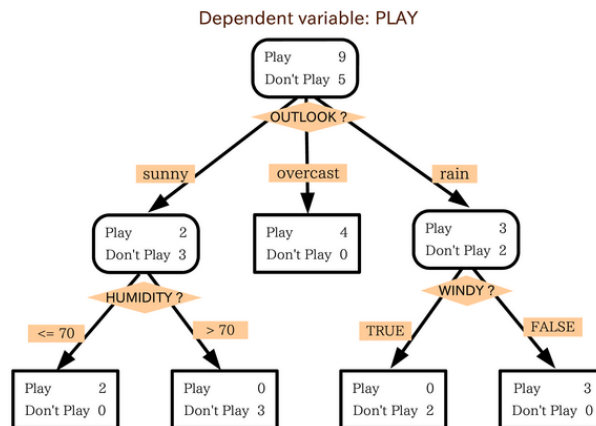


Figure 2: Decision tree example

By T-kita at English Wikipedia - Transferred from en.wikipedia to Commons., Public Domain, <https://commons.wikimedia.org/w/index.php?curid=3442362>

This is an example of how a normal tree looks like where we decide if we should play or not. We have 9 tuples/instances for playing and 5 for not playing. We see that for each level we get lower entropy, or in other words: it contains more and more of one class and fewer of another. This happens until we reach the bottom where we have pure leaf nodes.

4.3 Neural Network

Two hidden-layer MLP.

The neural network is the most well known of the methods for creating an AI. This uses several layers to come up with a usable model. There is one input layer, several hidden layers and then an output layer. The input layer is the data we want to work

with, and the output is the result of the model. This means that the hidden layers do the actual work of the model. These models use a lot of different techniques to become more efficient. To build a fast but flexible neural network we need to have hidden layers where are nodes called neurons. These have an assigned bias, and the input to these nodes have weights. The inputs are either the input nodes or other hidden layers. This is fairly limited and would only provide linear classification, but with activation functions we can give our model the ability to non-linear classify. [2]

4.3.1 Activation functions

These functions enable the models to become better than simple regression models. These introduce a form of non-linearity to the output of nodes. This is essential for a good model. A model could have as many hidden layers and neurons, but only perform as well as a linear model without the help of an activation function. An example of an activation function [3]

4.3.2 The model for this project

There were two neural network models used during this project. One was made by sci-kit learn and the other was made with pytorch. The one made by sci kit learn was made for testing the performance of the neural networks with the built in functions for F1 scores, AUC and recall. See more in section 4.5.3 and below for further info about the metrics. The one made with pytorch was made for testing of the integrated gradients method, where the main library only accepts tensor objects, something that the Sklearn library do not provide as standard. This was fixed and there is one model. The one made by pytorch as the conversion from the results could easily be converted to tensor objects and eliminated the reason for having two models.

4.4 Training Setup

- 75/25 train-test split
- Stratified sampling
- Standardization

A training split is what we do for supervised models. It helps us correctly train the model for classification problems. The events are well defined and there is no need for a model to define its own events as it is for unsupervised training.

This could be something to look into for a different project. The unsupervised models are good at finding patterns and problems that we humans do not find easily.

Stratified sampling is a method in statistics to not minimize errors when sampling, where the sampler is not extracting random samples from the entire population but rather from subgroups. [17]

Standardization is a common method used to make it easier for a model to learn. A lot of estimators have a great use for datasets where features look like standard normally distributed data or data that looks like gaussian data with 0 mean and unit variance. From sklearn documentation we can learn that the formula for standard scaler is:

$$z = (x - u)/s \tag{11}$$

Where the input x is subtracted by the mean of the training samples u and divided by the standard deviation s . [13]

4.5 Performance Results

Model	AUC	F1	precision
LR	0.99	1	1
RF	1	1	1
MLP	0.99	1	1

Table 1: Model comparison

We look at the weighted average. These values are all valuable metrics to gauge how well the model performs.

4.5.1 Problems encountered

The first training sets featured was had some mistakes that clouded the models judgment. The training set was made in two steps. The first step was to create a dataset with various features over a set time period. These features will have some spikes in usage. This is described more in detail above. The second step is what caused the problems. Here we use the baseline detect program. This detected some events, but not all. The training set was essentially useless with these baseline detections and contributed to the AI models performing guesswork. This mistake took went unnoticed for a period of time, because of the focus on the weighted average and not the macro average. It was not until the creation the AUC score of the MLP was created, and it had an AUC score of 0.5, which is the same as guessing the result. In the process of trying to improve this score it was discovered that the models did in fact not do anything other than set the result to a non event.

Since the events or rather what we want to classify is a somewhat rare occurrence, the scores achieved showed a high score when it came to the weighted average, but when it came to the macro average it was around 0.5. This was for precision, recall and f1 score, while the AUC score remained high. This shows that the models only correctly identified non-events and completely disregarded the events. The only model that was able to guess some events correctly was the random forest.

This was eventually solved when the target label used for training was changed from "is anomaly" from the baseline detections to "is event" from the synthetic data creation. This proved to be a good decision, as the macro scores came near the weighted average.

These scores were tested with an relatively easy set with low noise.

Model	Recall	F1	Precision
New LR	0.99	0.99	0.99
New RF	1	1	1
New MLP	0.99	0.99	0.99
Old LR	0.50	0.49	0.49
Old RF	0.65	0.71	0.87
Old MLP	0.50	0.49	0.49

Table 2: Macro avg across traning sets

4.5.2 Too high scores

Most models that are trained with "is event" and not "is anom" become incredible at deciphering the right class. This raises concern that there is a leak somewhere. An AUC score of 1 is a perfect score where the model did not make any mistakes. After testing with different noise levels, the conclusion is that the models are just very accurate when it comes classifying the different training data provided. The models were then tested with increasing levels of entropy/noise. The scores then dropped some, but were still in the 0.9 - 0.8 mark.

4.5.3 F1 score

The F1 score is one of the most used scores in evaluating machine learning models

$$F1 = \frac{2 * TP}{2 * TP + FP + FN} \quad (12)$$

TP is the number of true positives, FP the number of false positives and then FN is the number of false negatives. This describes how the F1 score is calculated, and gives us an understanding of what it is. This is the harmonic mean of precision and recall. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html <https://encord.com/blog/f1-score-in-machine-learning/>

4.5.4 Precision

$$Prescion = \frac{TPR}{TPR + FPR} \quad (13)$$

The ability of a model to tell whether an input value is an event or not. Best value is 1 and worst is 0.

4.5.5 Recall

$$Recall = \frac{tp}{tp + fn} \quad (14)$$

Recall or sensetivity is used to find out how many off the positives values were correctly assigned. Best value is 1 and worst is 0.

4.5.6 AUC

https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc#area_under_the_curve_auc The area under the curve score comes from the receiver operating characteristic curve (ROC). The ROC is made by looking at every possible threshold for classification. The curve is contained inside a box where each side is 1 in length. The y axis is the TPR or true positive rate and the x axis is made up of the false positive rate FPR.

$$TPR = \frac{\text{True positives}}{\text{True positives} + \text{False negatives}} \quad (15)$$

$$FPR = \frac{\text{False positives}}{\text{False positives} + \text{True negatives}} \quad (16)$$

An example, the classification of spam email (binary classification): There is a threshold that can be moved from 0.0 to 1.0. Then the threshold decides if the mail is spam or not. The threshold starts at 0.0 and the TPR rate is 1.0 and the FPR is 1.0. The TPR is 1.0 since it correctly identifies all the spam emails as being spam emails, but the FPR is also 1.0, this is because of threshold identified every non spam-email as spam. The process of finding thresholds that introduces change continues until we reach a threshold of 1.0, and both TPR and FPR is equal to zero. The curve made from this process is then used to get the AUC score. This is the area under the curve. The bigger the area the better the model. It also gives us information about what the best threshold for the model is. This is the point closest to the point (0,1). This gives the best performance with the highest TPR and lowest FPR.

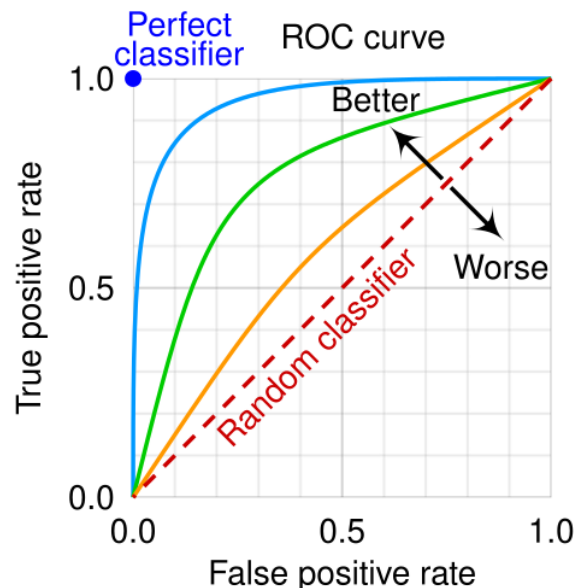


Figure 3: ROC example

By cmglee, MartinThoma - Roc-draft-xkcd-style.svg, CC BY-SA 4.0,
<https://commons.wikimedia.org/w/index.php?curid=109730045>

5 Explainable AI Methods

5.1 The need for explainable AI

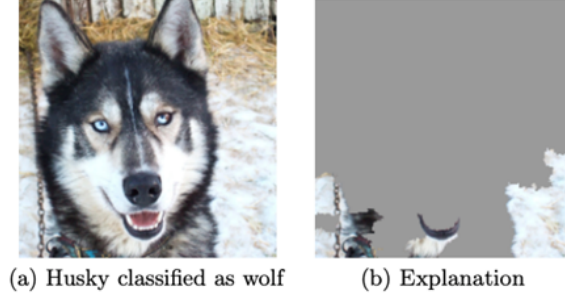


Figure 4: Example of bad interpretation

[7] The picture above is an example of how an explanation of the result can give insight into why a classification might not be correct even though the actual result is close or even correct. This picture shows that the AI model uses the snow as a reasoning for this classification. This can occur for models where they "look at the wrong places". The explanation reveals that the model focuses on the wrong features, in this case the pixels are features. The model should have based its explanation on the dog itself and not the snow. The explainable AI helped in showing the user why it should/shouldn't trust the model.

5.2 Permutation Feature Importance

Permutation Feature Importance is a simple method for understanding the importance of a feature in a model. First we calculate the performance of a model with a training set so that we know how well it is supposed to work. Then we go through each of the features. The first feature is shuffled or permuted. The model is then tested with the new dataset, and the performance drop is noted. The dataset is reverted to before the change. The second feature is then tested in the same way, and this keeps going until every feature is tested. How much the performance drops gives us an understanding of how important a feature is. The value of feature goes up by how much the performance goes down. This is a global method that gives a result for the whole method. There is no explanation for a simple instance. [12]

This formula is how sklearn implements it, and the algorithm goes as follows:

- Inputs: fitted model m and tabular dataset(validated) D
- The s is the base score calculated from the m and the set D without any permutation (This project uses $f1$ as the score).
- For every feature j :
 - For each repetition k in $1...K$:
 - * Shuffle the feature j (column j in D) randomly to corrupt the data into $\tilde{D}_{jk}$

- * The score s_{kj} is calculated from the corrupted dataset $\tilde{D}_{jk}$
- then finally compute the importance i_j for feature f_j as:

$$i_j = s - \frac{1}{K} \sum_{k=1}^K s_{k,j} \quad (17)$$

[12]

5.3 SHAP

Global importance(mean):

$$Global_j = \frac{1}{N} \sum_{i=1}^N |\phi_{i,j}| \quad (18)$$

Shap is one of the most important explainable AI in the field of AI. It uses game theory to determine different ways to explain the outcome of a model.

SHAP uses Shaply values from game theory. Shapely values were introduced by the mathematician Lloyd Shapley. These determine how each input is credited or blamed for the decisions. "Who contributed the most?", or "who contributed the least?", and "who ruined the result for the others?" are some of the questions that are answered with shapely values. An example of this can be a team game tournament where the players are the features and their outcome help us determine how well each of the players do. We want to see how every player work alone, work with others and how they work against each other. To do this we need to see every single permutation/subset of the features. Only after we have gone through every possible configurations will we be able to see the true shapely values for the features.

There are multiple ways to describe the outcome of SHAP with plots, some look at the specific tuples in the outcome to give a more detailed description, while others help us to look at the entire outcome as a whole. For example a bee swarm plot:

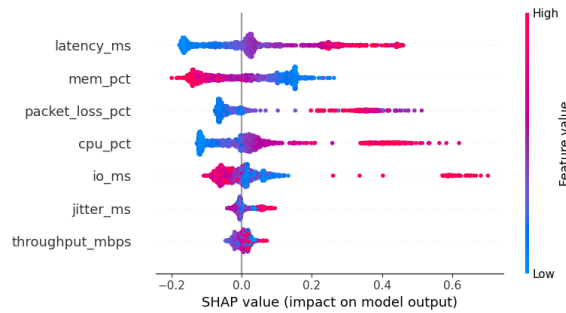


Figure 5: SHAP summary

Here we see the why it see the feature values as red or blue depending on how high the actual values are. From the diagram we can see that some very high I/O delays contributed a lot, but all the other values were fairly low value, and did not contribute much.

Shap is the slowest of the methods discussed here, that is because it uses all the different permutations of the training set. To give a good explanation, we need to see all

of the subsets of the training set. If we remove CPU from the training set then how do the rest of the training set do. We remove features and look at all the different subsets to gain a deep understanding for how the features work with each other. The amount of available subsets grow at an exponential rate, but methods like tree explainer helps us calculate at a polynomial time instead of exponential time.

The equation for feature importance of a single feature:

$$\phi_i(v, N) = \sum_{S \subseteq N \setminus i} \frac{|S|!(n - |S| - 1)!}{n!} (v(S \cup i) - v(S)) \quad (19)$$

[16]

This is the calculation for the shapely value of one feature in the model. Where the input is N the total number of "players" or features, and v is a black box method. The calculation takes all the subsets of N with the i-th feature removed. This is for global features, but there are equations that look very similar that are made for specific data points. This equation looks like:

$$\phi_i(f, x) = \sum_{z' \subseteq x'} \frac{|z'|!(M - |z'| - 1)!}{M!} [f_x(z') - f_x(z' \setminus i)] \quad (20)$$

[8]

- The ϕ_i signifies the feature i.
- The input values are the black box model f and an input value x.
- The f is the model we want reason about
- x is a datapoint with real values, and is not an abstract value for this equation. The same way N is for the more general one.
- The z' is a subset of all the features of a simplified x'. It is simplified since we might not need all features. For taking every pixel in a picture is too big and we need to do some grouping to simplify.
- We will loop through every subset to see the significance off i feature. The subsets also include the empty subset.
- $\frac{|z'|!(M - |z'| - 1)!}{M!}$ signals the weight of the contribution. The more features in the subset the bigger the weight, but also the times there are few features the weight gets big.
- The contribution is $[f_x(z') - f_x(z' \setminus i)]$. Here the contribution comes in the form of seeing the difference between the performance of between the model with and without the feature i. Then we see if the probability of an event happening increases or decreases. This the contribution or the marginal value.
- Example: If i is the throughput feature with value 85. And the subset z' is the features cpu_pct with value 60 and io_ms with value 5.7. Then we can see that the probability increases with i as the throughput is much more central to events such as congestion.

5.4 LIME

Local linear surrogate:

$$\hat{f}(x) = w^T x \quad (21)$$

Lime stands for Local Interpretable Model-agnostic Explanation. Lime is primarily a local explanation where we look at a smaller part of the model to determine behavior. We take a small part of the model and look out how it compares to a single instance. This is a lot faster than other methods. It is also made to be Interpretable by humans, to give us a good understanding in a readable format. Model-agnostic means that can be used on any model which gives it an advantage in usability. The LIME explainable AI can be divided up in 4 steps:

1. LIME chooses a local part of the model. This is done by selecting an instance and looking at a small part around it. The distances are so small that a non-linear model becomes linear or near linear, from here we can interpret the black box prediction.
2. In the next step LIME perturb or disrupt the training set by placing new points close to the selected instance. These will be called dummy points for the rest of the report.
3. Next we weight the samples by proximity to our selected instance.
4. LIME gives an explanation that is understandable for humans.

[5]

To get a better understanding we can look at this simple 2D picture of a model. The dotted line is the new linear model, and the dots and crosses are the dummy points made to see what features subtracts and what adds to the final decision. The symbols get smaller and smaller the further away from the center of the local model. The features closer are weighted more than what the features further away are.

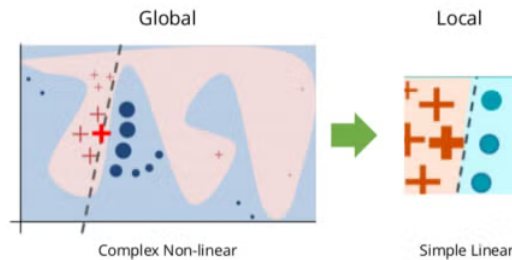


Figure 6: LIME example

[11]

The limitations: Only looks at one instance, and can therefore change by where we set our instance. But this is not only for instances that are far apart, but slight changes in instances have also proved to be able to change quite a bit. This has to do with the picture or information given to LIME. Since we approximate our model to get a more linear one, which we can reason about. This can make the linear model change a lot from instance to instance if the real model is heavily non-linear in some parts.

5.5 Integrated Gradients

$$IG_i(x) = (x_i - x'_i) \int_0^1 \frac{\partial F(x' + \alpha(x - x'))}{\partial x_i} d\alpha \quad (22)$$

Integrated gradients are an efficient way to calculate the worth of every feature in the tuple. It is a very adaptable equation that can be used in a large group of models. It can be used in everything from images, text and structured data. It is mostly used in for models within the field of images. Integrated gradients are mostly used at to calculate the differentiable models instead of non-differentiable. Examples off differentiable models can be models like neural networks or logistic regression. These are the models most often used in modern machine learning, especially the neural networks for large language models. This is a gradient based explanation method. This is simply a method that maps the inputs/features to the output. In other words it gives an explanation by the gradients, the change from input to output. The above formula is the general idea but computing a definite integral can be an expensive computation, so we take a page from digital signal processing and calculate with discrete values.

$$IntegratedGradients_i^{approx}(x) ::= (x_i - x'_i) * \sum_{k=1}^m \frac{\partial F(x' + \frac{k}{m}(x - x'))}{\partial x_i} * \frac{1}{m} \quad (23)$$

- The i is the feature
- The x is the input
- The x' is a baseline to compare x with.
- The partial differentiation equation is written as $\frac{\partial F}{\partial x_i}$. This means that we differentiate by x_i
- k = scaled feature perturbation constant
- m is the amount of steps in the Rieman sum approximation of the integral.
- $x_i - x'_i$ is a scaler
- $x - x'$ is the difference between the baseline and the feature, a constant.

The $x - x'$ is the gradient. The easiest example is to imagine the features as the pixels of an image, and the baseline as a black image(or noise, gaussian distribution or random values of the picture). The black image is then the baseline x' , and the picture becomes x . The difference between these two pictures is then calculated. For tabular data the black image is the equivalent to zero values.

There are multiple steps m that we take. This is the steps from going from the baseline to the input. Calculating the gradient and taking the average of the gradients.

$$\sum_{k=1}^m \frac{\partial F(x' + \frac{k}{m}(x - x'))}{\partial x_i} * \frac{1}{m} \quad (24)$$

The usual amount of steps are $m = 50$. This is then multiplied by the difference between the baseline and the input: $(x_i - x'_i)$. [10]

5.6 Strengths and weaknesses

Most of the explainable AI that we have discussed here are model agnostic. This means that they can give a good explanation for every model. Integrated gradients is in practice not model agnostic as it requires the model to be a differentiable model. All explainable methods are viable. The use case can vary and needs to be looked at. SHAP might be the most comprehensive, but struggles with speed when it comes to more complex models. It has several methods to explain different models. An example is the tree explainer which can explain random forest. This is something that other models can have problems with. The other models are faster and can give good explanations for a single event, but might be less reliable and accurate. It performs only as well as the model it uses.

6 Evaluation Methodology

6.1 Top-k Overlap

$$Overlap@k(A, B) = \frac{|Top_k^A \cap Top_k^B|}{k} \quad (25)$$

Finds the similarities of overlap between two methods.

6.2 Driver Recovery@k

$$DR@k = \frac{|Top_k \cap D(e)|}{|D(e)|} \quad (26)$$

A simple equation where we check how many of the top K attributes are in the expected set.

6.3 Stability Under Noise

Add Gaussian perturbation:

$$x'_i = x_i + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2) \quad (27)$$

Some explainable AI do better than others when it comes to noise. Therefore the both the models and the explainable AI were tested with datasets with higher noise. Nothing of importance was found.

7 Experimental Results

7.0.1 Models used for testing

The models used were mainly Logistic Regression and Random Forest with the exception of using Neural networks in Integrated gradients as this is the easiest. The reasoning for this is that the only good method that is already made uses tensor objects and revolves around the torch library for making AI. The argument could be made that we could simply make our own Integrated gradient that is more compatible with the random forest and logistic regression.

7.1 Global Importance Comparison

As mentioned earlier the Permutation Feature Importance method is the only method that is purely focused on the global importance as it can not produce any meaningful explanation for single instances.

All the methods were tested with the same easy synthetic data set that span one day, measured every minute. The noise was rather low.

Shap Feature ranking	Shap scores	PFI feature ranking	PFI scores
1. Latency	0.033	1. Latency	0.239
2. Packet loss	0.023	2. CPU usage	0.207
3. CPU usage	0.021	3. Memory usage	0.146
4. io delay	0.009	4. Packet loss	0.137
5. memory usage	0.008	5. IO delay	0.1064
6. Jitter	0.005	6. Throughput	0.000
7. Throughput	0.002	7. Jitter	0.000

Table 3: SHAP and PFI global importance

LIME Feature ranking	LIME scores	IG feature ranking	IG scores
1. Memory usage	0.306	1. Memory usage	1.269
2. latency	0.256	2. Latency	0.6281
3. Packet loss	0.128	3. CPU usage	0.593
4. CPU usage	0.125	4. Throughput	0.122
5. IO delay	0.073	5. IO delay	0.063
6. Throughput	0.056	6. Packet loss	0.006
7. Jitter	0.037	7. Jitter	0.003

Table 4: Lime and IG global importance

The reason for the amount of separation in this table is because of the difference in scoring when it comes to the four methods. PFI uses the f1 scoring method for detecting.

The scoring for PFI shows how the equation works with an example, each of the features are perturbed and then it is tested to see if the performance drops. The F1 scoring method is used for in this example. PFI then gives scores of how important the feature is based on how bad the F1 score was when that feature was perturbed and its base score when none of the features are perturbed. The scores show that both jitter and throughput have nothing to do with how well the model manages to classify. The other models do not offer this insight since they are mainly local explanations where the mean is used on all the local explanations. This causes the features that are irrelevant to the global state to get a score higher than zero, but we can still see that Jitter and throughput are usually the lower scores.

For these test there were only used one AI model for each of the explainable AI. Shap and PFI used random forest, Lime used Logistic regression and Integrated Gradients used a neural network made from pythorch.

I will display the difference that a model has on the explainable AI with the use of LIME.

LIME(Logistic regression)	Scores	LIME(Random Forest)	Scores
1. Memory usage	0.306	1. Packet loss	0.130
2. latency	0.256	2. Latency	0.129
3. Packet loss	0.128	3. Memory usage	0.113
4. CPU usage	0.125	4. CPU usage	0.109
5. I/O delay	0.073	5. I/O delay	0.069
6. Throughput	0.056	6. Throughput	0.017
7. Jitter	0.037	7. Jitter	0.007

Table 5: Lime difference in model performance

Here we can see the differences clearly in the scores given. When Lime was used with the results of the logistic regression it gives a clear weight advantage to the first 2 attributes: memory usage and latency, and the rankings are clearer. While the results from LIME using random forest is more evenly spread, with the four at the top of the ranking having almost equal say.

7.2 Local Explanation Analysis

Each of the three methods supplies their own way to give an explanation for a row or instance. There are only three since PFI does not give a local explanation, as discussed before.

SHAP uses Explanation objects. These are objects that can be a bit annoying to use properly at first, this comes from the complexity of them. There are a lot of functions and data, that a user might need to work with a bit to get the result that is wanted, but after the explanation object is understood by looking at the documentation, it will offer a wide range of ways to understand the explanation. This is either by looking at the text output or by looking at the many images that the objects can create. These images can give both global and local explanations.

The Lime objects do their own ordering and ranks the attributes in the instances based on contribution. It also gives an explanation in form of a string. This makes it easy for user to use it directly as it is ordered highest to lowest. It also gives an explanation of how LIME sets up the data to perform its test, as it gives an range for the perturbation of the data used for surrogate model.

The IG gives the most bare bones answer with only the

7.3 Driver Recovery Results

7.3.1 Pre processing

The way this calculated is via a special ordering. The Lime local explanations orders the objects into a ranking which is not compatible with this method. This needed to be re-ordered into where the columns followed the feature order. So in the end of the xai.LIME code, there is a costly, simple, multi order loop to re order the data to be correct. This is the order that both of SHAP output and Integrated gradients follow. It is now easy to process the data of all the local explainable AIs in the driver recovery in the same way.

7.3.2 Results

Event	PFI	SHAP	LIME	IG
Congestion	-	0.74	0.91	0.68
CPU	-	0.99	0.98	1.00
IO	-	0.72	0.70	1.00

Table 6: Driver Recovery@3

The PFI can not be tested via this way as it is purely global, and can not be tested in the way that we have tested the other ones. The other methods can create local explanations where only one instance is tested. It is impossible for PFI to only describe a single instance in a dataset. As discussed earlier, the PFI algorithm takes a single feature at a time and scrambles the that column a number of times. It is generates test scores from these test. The worse the score the more important a feature is. This does not work with one row of data and needs a big set.

Now how do they do when there is added noise:

Event	PFI	SHAP	LIME	IG
Congestion	-	0.687	0.783	0.690
CPU	-	0.960	0.944	1.000
IO	-	0.571	0.667	0.960

Table 7: Driver Recovery@3 with noise

Shap on average dropped 0.077 in score. Lime dropped 0.0653 on average, while IG only dropped 0.01 with its score for congestion increasing.

The highst average score under noise is: 0.883 from Integrated gradients followed by LIME with a score of 0.798, and SHAP with the score of 0.739.

This is a surprising find, as it highlights that IG for the simple neural network we build performs the best over noise change.

8 Real life data

8.1 From synthetic to real data

The conversion from synthetic to real data discovered some problems with the training data. The first problem is how to measure the jitter, latency and package loss. It was solved using the sub process library, which lets the programmer use the command line and writing the output to a text file. Using this we could use the ping function to send an arbitrary amount of pings to a random address. For this program the amount was 50 and the address was google.com. The output was then sent to a text file. The second problem was the I/O delay, which has been previously discussed on how it is hard to measure. It turns out that the previous problem solved this problem. As after we write to the text file we need a way to read from the file. This is where the I/O delay come in. The time is taken from reading the whole file. The argument could be made that the

I/O delay should only be read after reading zero bytes, but for such small files the time taken to read is not going to be a great contributing factor. The Latency is the average of the round trip time for these ping packages. The Jitter is calculated as:

$$J = \sqrt{\frac{1}{N} \sum_{i=1}^N (D_i - D)^2} \quad (28)$$

[6]

Another problem was the use of root access. This is one of the core problems that would make it hard to use on any server application. This data collection can not use any form for functions that require root privilege. This is why the delay on the pings are only 0.2 seconds since any lower requires root access. There were also some ping libraries that only worked with root access, these could not be used. The only way that the program now needs any privileged access is if the user that runs the program does not have access to either read or write in the folder the program runs in.

8.2 Comparison of data

The dataset is around 40 entries collected on a desktop with fiber ethernet, a ssd, 64 GB ddr4 memory and an AM4 cpu. The data is not similar to the synthetic, The . The throughput has a much higher value. Where the average value is usually around 150, the real values are anything from 300 to 800. The I/O delay is much faster, as it was measured on an ssd. The synthetic data for I/O delay is some few milliseconds, but for the ssd it is around 10^{-5} as big. There is no packet loss in the real data set as the connection during the time period, there were no problems with the internet and the ping message was sent to a safe address that is used to a lot of data. The location was "google.com". With 50 pings being sent for each instance.

8.3 Real data experiment

8.3.1 The initial test with models

The test was conducted with random forest and linear regression as the AI models. The models themselves performed completely different. The linear regression flagged every instance as an event, where it gave every instance a value of of 99% or higher. The random forest flagged none of the instances as events, which is correct, as nothing major happened while taking the samples. To see why they came to these explanations we need to use explainable AI.

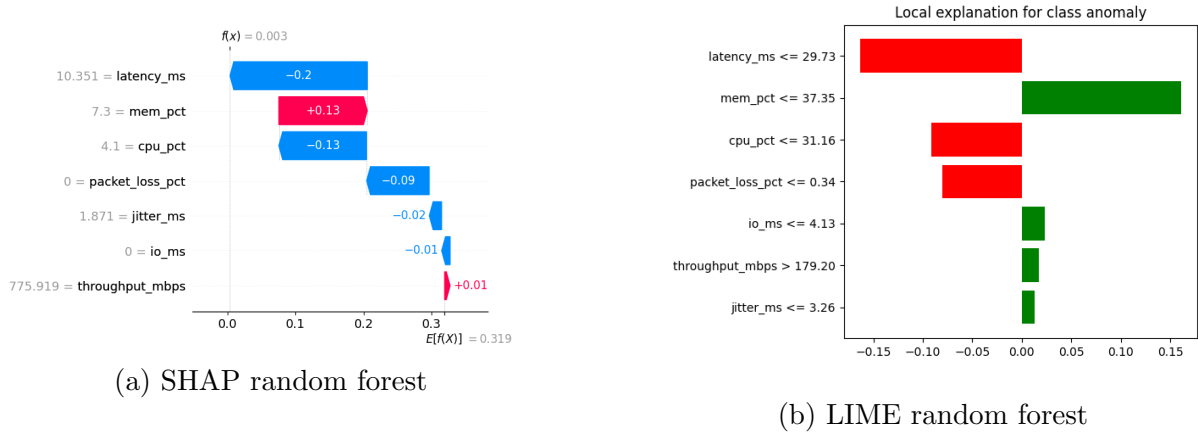
8.3.2 The explainable AI setup

The explainable AI used are SHAP and LIME as these are the most popular and used. The methods used in SHAP and LIME are also very different compared to each other. SHAP looks at every subset and finds synergies and conflicts, while LIME uses surrogate models. This test will also highlight some of the weaknesses of the LIME model when presented with a much different data set then what it was trained on, and which do not contain any events. At least not any events that can be seen by a human. This also displays the strength of SHAP.

8.3.3 The results

Firs is the random forest model, which guessed correctly that none of the instances were event. This gives us a baseline. To compare the logistic regression model that flagged everything as an event. For this experiment, the most important is the explanations for the instances as these describe what really went wrong. The global descriptors are also interesting to see what the average main drivers are, but for this experiment we will only look at how good a xai model is in delivering good information for an instance. For this experiment the we chose an arbitrary index(5) in the real dataset. The logistic regression model gave a 100% certainty that this was an event, and the random forest gave a 22% chance.

8.3.4 The results for random forest



Shap shows us why this is not an event, with a base value of 0.319. The plot displays the drivers for the choice

This is The LIME explanation, which is for the most part exactly the same. The only difference is that this only show the relative weights of the features. The

8.3.5 The results for logistic regression

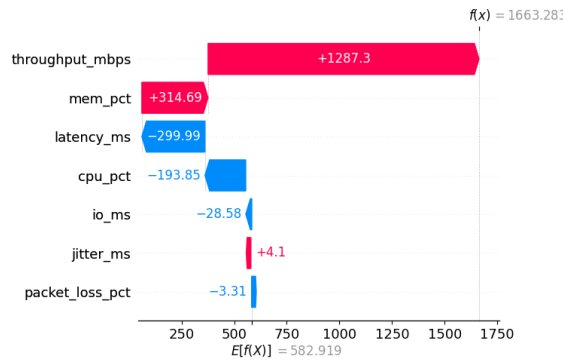


Figure 8: SHAP logistic regression

From the results of the model, this is the expected result. SHAP displays the reason for why logistic regression flagged the instance as an event. The throughput was exceedingly higher than normal and therefore it pulls the result to be an overwhelming true value.

The rest of the values are also higher than normal, with their contribution. This could be because of the high value in throughput. This could cause the synergies and hindrances to increase in value also. They are still a small fraction of the contribution to the total output compared to throughput.

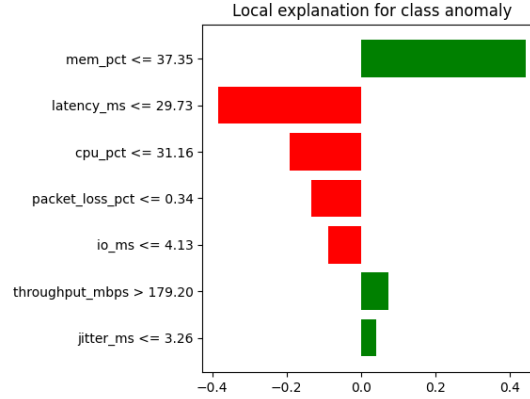


Figure 9: LIME logistic regression

This is the result that causes the most confusion. As it shows something completely different than what was shown in SHAP. For reference is this the result for an anomaly in the training set that uses LIME and logistic regression:

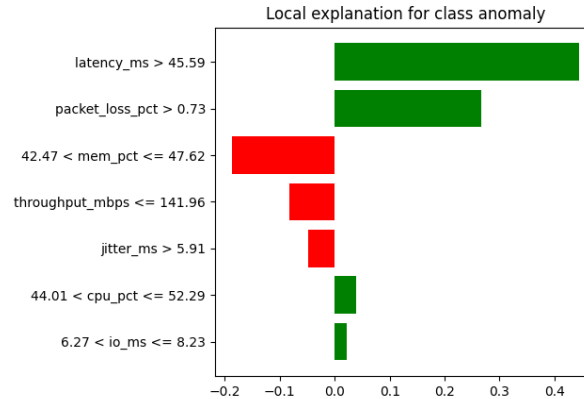


Figure 10: LIME logistic regression real event

The explanation for this could be many things. One reason could be a mistake in the testing, but the only difference is the dataset used. Another difference could be a failure with LIME, and not a implementation mistake. LIME is known as very sporadic, and uses a surrogate model. This surrogate model is supposed to dumb down the original model so that it can then place some dummy points that give insight to the model. The problem is the instances that will be the center for these dummy points is so distant from being the deciding factor in the classification, causing none of features to have any real

impact on the decision in the eyes of LIME and its surrogate model. This would likely not happen to the random forest algorithm as it is more stable and is a lot less influenced by a single attribute, causing the surrogate model to operate in reasonable place where the dummy points matter.

A simple example of how this is not working in LIME with logistic regression. The Line is the complex model drawn in 2D to make it easier to reason with. The black circle at the top is the surrogate model where most of the dummy points lie, with the instance in the middle. There are other dummy points as the smaller circles, but these are so far out that the results of them do not matter. The surrogate model have do not interact with the real model in any meaningfull way, and therefore give no information to the user.

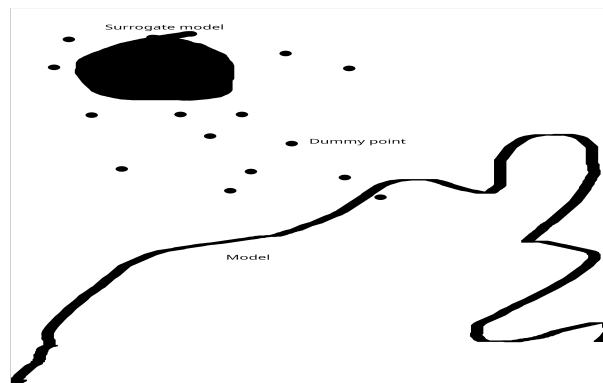


Figure 11: Example of LIME

6

8.3.6 Conclusion for test

The random forest is much more robust than the logistic regression model. SHAP performed well, and as expected from the results that the models gave, while LIME works well if the surrogate models it creates is close by the actual model.

9 Conclusion

This project demonstrates that:

- Synthetic ground-truth enables objective evaluation.
- SHAP and IG show strong driver recovery.
- LIME exhibits higher variance and can make mistakes.

Future work includes applying the framework to real monitoring data and extending evaluation to adversarial robustness. The explainable AI methods all showed great potential in the field of AI for network monitoring.

Further testing and developing needs to be done in order to assure that both the model works, and that the explainable AI can also be trusted. Which is similar to how baseline detections was trusted in the start, but after further testing became a liability.

The most robust was SHAP as it has a can explain almost anything through a stable algorithm. PFI is method that does well with lots of data, but lacks any form of method for giving a good local explanation. The IG is a method that performs best on pictures, but also does well for tabluar data as well as the other xai methods. LIME is fast and can give a good explanation if it the model and inputs are not too far apart. A good combination of all these methods would provide a good background for knowledge to the ones who should use it.

The synthetic dataset provided a fair basis in which we could reason about both the models and the explainable AI methods. The reason that it was good is that this was to show how it could be done, not a definitive way of doing it. The users and companies using xai should build more robust models that encompasses more features and events.

The real dataset which was tested upon should have included events. The real dataset should been over a longer period of time with events, but these events do not happen at a whim unfortunately.

This concludes the paper.

References

- [1] Dr. Mathias Jesussek[numiqo] Dr. Hannah Volk-Jesussek. *Logistic Regression [Simply explained]*. 2023. URL: <https://www.youtube.com/watch?v=C5268D9t9Ak>.
- [2] Google.com. [Online; accessed 15-May-2026]. 2025. URL: <https://developers.google.com/machine-learning/crash-course/neural-networks/nodes-hidden-layers>.
- [3] Google.com. [Online; accessed 15-May-2026]. 2025. URL: <https://developers.google.com/machine-learning/crash-course/neural-networks/activation-functions>.
- [4] Chriss Hoffman. *Why It's Good That Your Computer's RAM Is Full*. [Last accessed 12-may-2026]. 2016. URL: <https://www.howtogeek.com/128130/htg-explains-why-its-good-that-your-computers-ram-is-full/>.
- [5] Aleix Nieto Juscafresa. *Understanding LIME , Explainable AI*. 2022. URL: <https://www.youtube.com/watch?v=CY1172IwqKs>.
- [6] Alyssa Lamberti. “How to Measure Jitter Keep Your Network Jitterbug Free”. In: *Obiko blog* (2023). URL: <https://obkio.com/blog/how-to-measure-jitter/>.
- [7] Oge Marques. “Explainable AI (XAI): Implement explainability in your work”. In: *Matlab* (2023). URL: <https://blogs.mathworks.com/deep-learning/2023/06/27/explainable-ai-xai-implement-explainability-in-your-work/>.
- [8] Christoph Molnar[DeepFindr]. *Explainable AI explained! NR.4 SHAP*. 2021. URL: <https://www.youtube.com/watch?v=9haIOplEIGM&t=230s>.
- [9] [Normalized Nerd]. *Decision Tree Classification Clearly Explained!* 2021. URL: <https://www.youtube.com/watch?v=ZVR2Way4nwQ>.
- [10] Conor O’Sullivan. *Integrated Gradients Explained — Theory, Axioms Python Implementation*. [Online; accessed 15-May-2026]. 2026. URL: <https://www.youtube.com/watch?v=CpiX4WurL7w>.

- [11] Marco Túlio Ribeiro, Sameer Singh, and Carlos Guestrin. “Why Should I Trust You?”: Explaining the Predictions of Any Classifier”. In: *CoRR* abs/1602.04938 (2016). arXiv: 1602.04938. URL: <http://arxiv.org/abs/1602.04938>.
- [12] sklearn. *Permutation feature importance*. [Online; accessed 15-May-2026]. 2026. URL: http://scikit-learn.org/stable/modules/permutation_importance.html#permutation-feature-importance.
- [13] sklearn. *StandardScaler*. [Online; accessed 15-May-2026]. 2026. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>.
- [14] Wikipedia contributors. *Decision tree learning* — *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Decision_tree_learning&oldid=1350009898. [Online; accessed 15-May-2026]. 2026.
- [15] Wikipedia contributors. *Logistic regression* — *Wikipedia, The Free Encyclopedia*. [Online; accessed 15-May-2026]. 2026. URL: https://en.wikipedia.org/w/index.php?title=Logistic_regression&oldid=1343937623.
- [16] Wikipedia contributors. *Shapley value* — *Wikipedia, The Free Encyclopedia*. [Online; accessed 6-May-2026]. 2026. URL: https://en.wikipedia.org/w/index.php?title=Shapley_value&oldid=1349441895.
- [17] Wikipedia contributors. *Stratified sampling* — *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/w/index.php?title=Stratified_sampling&oldid=1303141865. [Online; accessed 15-May-2026]. 2025.